

Gestion du temps dans les applications réparties

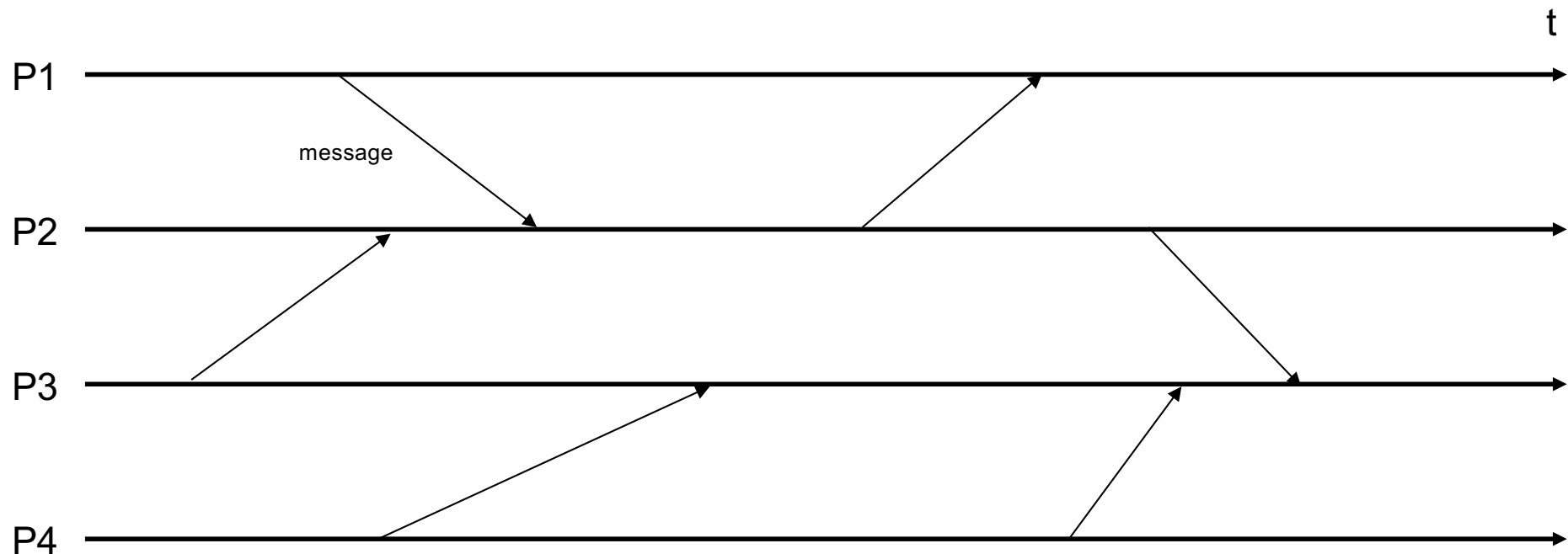
Master informatique
UE AR (MU4IN403)

Plan

- **Relation de précédence**
 - Principe de la causalité
 - Consistance des horloges
 - Forte consistance (caractérisation de la causalité)
- **Modèles d'horloges**
 - Scalaires (Lamport)
 - Vectorielles (Mattern / Fidge)
- **Implémentation d'horloges vectorielles**
 - Singhal et Kshemkalyani
- **Horloge matricielles**
 - Fisher et Michael
- **Causalité entre les messages**

Contexte (1)

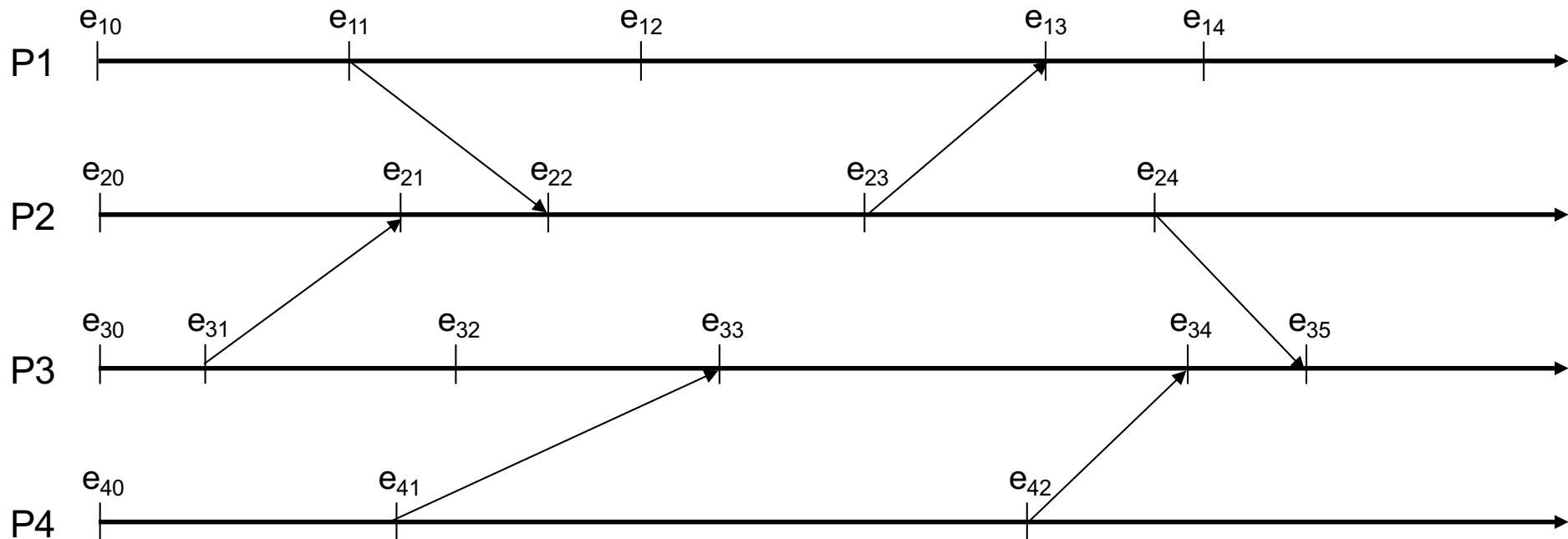
On considère un système réparti comme un ensemble de **processus asynchrones** s'exécutant sur différents sites. Pas d'horloge globale. Les processus communiquent **uniquement par message** (pas de mémoire commune). Les délais de communication sont finis, mais **non connus** et **fluctuants** (**modèle asynchrone**).



Contexte (2)

Pour étudier le système réparti, on s'intéresse à la succession des **événements** se produisant sur les différents processus. On définit trois types d'événements :

- les événements locaux (changement de l'état interne d'un processus)
- les émissions de message
- les réceptions de message



Le temps

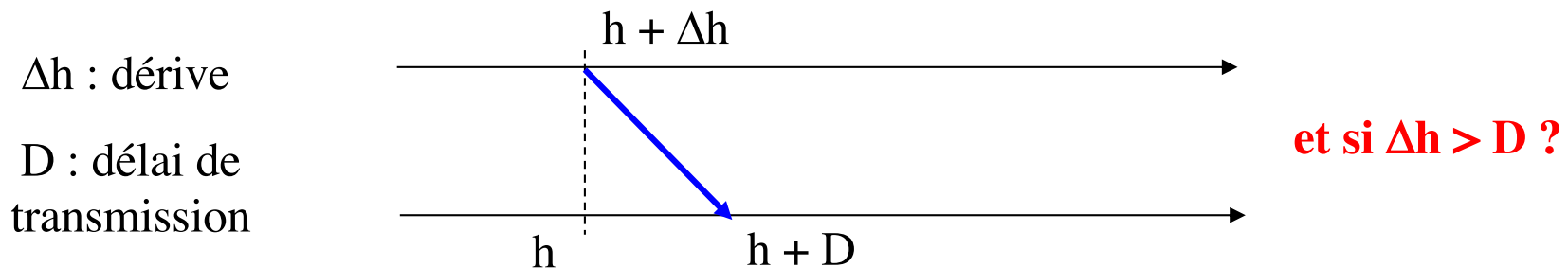
- **A-t-on besoin du temps ? On veut pouvoir**

- Tracer une exécution
 - debug

- ...

Relation de causalité entre événements

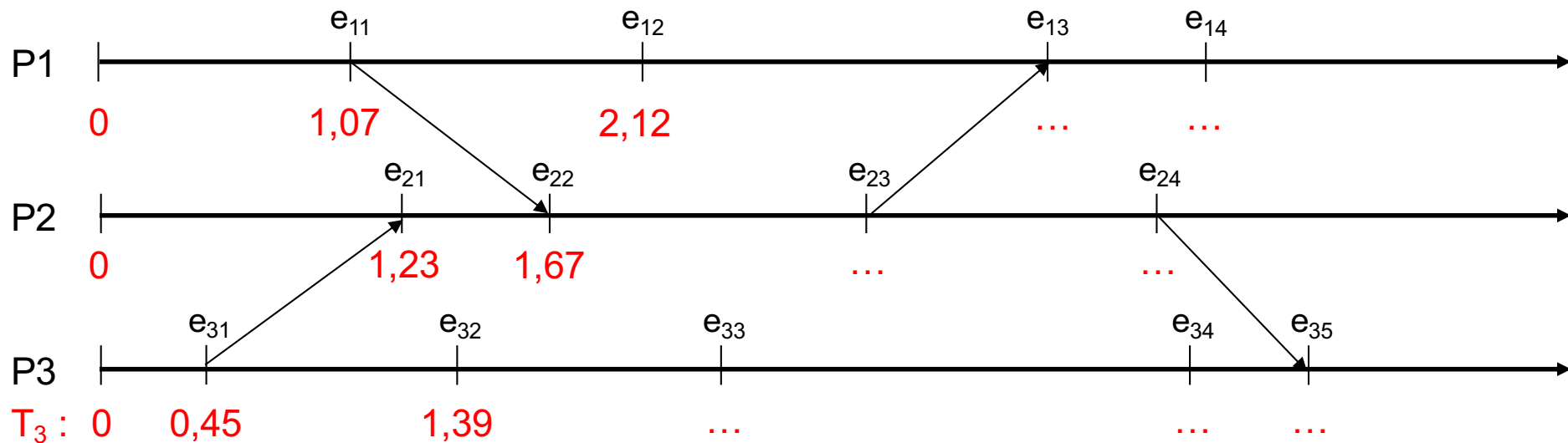
- **Pourquoi ne peut-on pas utiliser le temps physique ?**



Horloge Physique

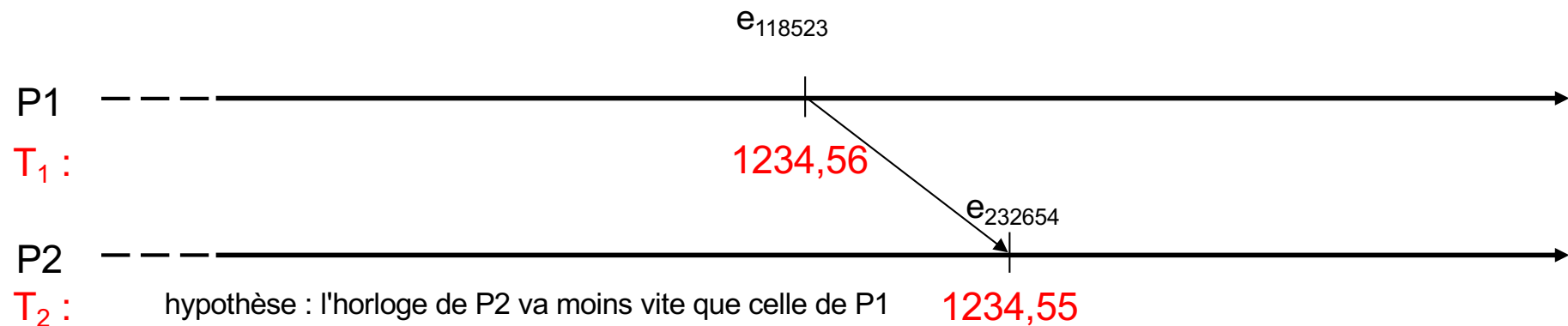
Les événements sont datés à l'aide de l'horloge physique de chaque site S_i . On suppose que :

- toutes les horloges sont parfaitement synchronisées au départ,
- leur résolution est suffisante pour dater distinctement tous les événements locaux.



Limites Horloges Physiques

Les horloges physiques dérivent au cours du temps, et le sens et l'amplitude de la dérive est propre à chaque machine. Les événements locaux restent correctement ordonnés par leur date, mais pas nécessairement ceux se produisant sur des machines distinctes. Par conséquent, la datation par horloge physique ne tient pas compte de la causalité des événements non locaux.



Conséquence : la précédence causale n'est pas respectée.

$e_{118523} \rightarrow e_{232654}$ et pourtant $T(e_{118523}) < T(e_{232654})$

Relation de Précédence : Causalité

Principe de la causalité : la cause précède l'effet.

Pour étudier la causalité entre les événements, on définit la relation de **précédence causale**, notée $\rightarrow$, traduisant une causalité **potentielle** entre les événements.

Définition

a $\rightarrow$ **b** si et seulement si :

- **a** et **b** sont des événements du **même processus** et **a** a lieu **avant b**
- **a** est l'**envoi** d'un message *m* et **b** est la **réception** de *m*
- il existe **c** tel que **a** $\rightarrow$ **c** et **c** $\rightarrow$ **b** (relation transitive)

$$a \rightarrow b$$

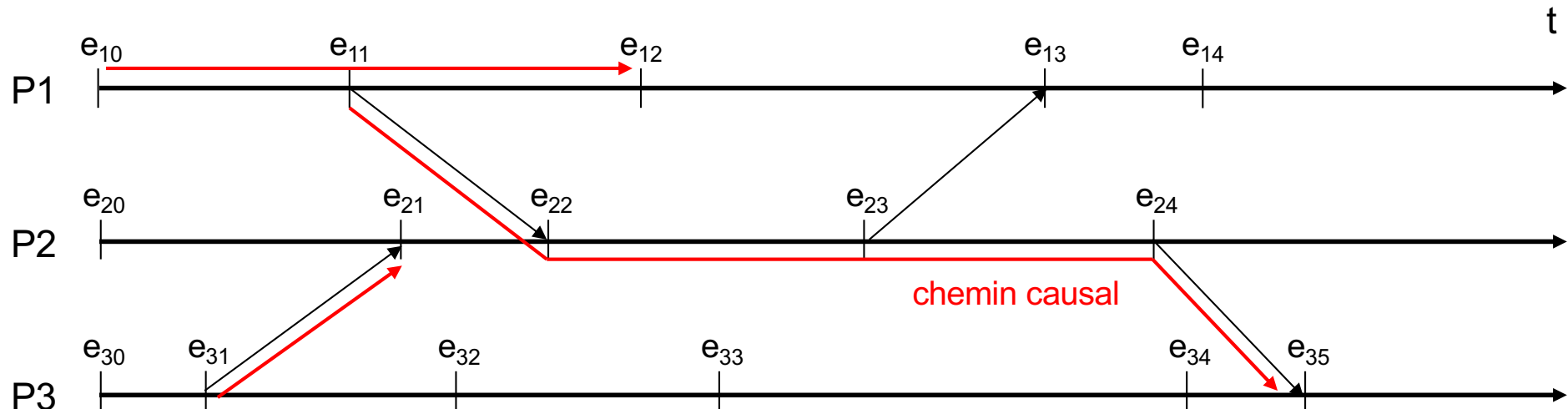
a a *potentiellement causé* **b**

a *précède causalement* **b**

a *happened before* **b**

Relation de Précédence: causalité

Deux événements qui ne sont pas causalement dépendants l'un de l'autre sont dits **concurrents**, noté $\parallel$: $e \parallel e' \Leftrightarrow e \not\rightarrow e'$ et $e' \not\rightarrow e$



Relations :

$$e_{10} \rightarrow e_{12}$$

$$e_{31} \rightarrow e_{21}$$

$$e_{11} \rightarrow e_{35}$$

$$e_{12} \parallel e_{22}$$

$$e_{32} \parallel e_{23}$$

$$e_{33} \parallel e_{14}$$

Précédence causale : $e \rightarrow f$

- e_i^n : $n^{\text{ème}}$ événement sur le site i

Definition (Lamport 78)

$e_i^n \rightarrow e_j^m$ (e_i^n précède causalement e_j^m) ssi :

- $i = j$ et $n < m$: **précédence locale**

ou

- $\exists \text{ mes}, e_i^n = \text{send}(\text{mes})$ et $e_j^m = \text{receive}(\text{mes})$: **précédence par message**

ou

- $\exists e_k^p, e_i^n \rightarrow e_k^p$ et $e_k^p \rightarrow e_j^m$: **relation transitive**

- l'ordre défini est **partiel** (existence d'événements $\parallel$)

Datation

A chaque événement on associe sa date d'occurrence :

$$a \mapsto_D D(a)$$

Objectif : trouver un système de datation D qui **respecte** et **représente** au mieux la relation de précédente causale.

- respecte : $(a \rightarrow b) \Rightarrow D(a) < D(b)$ exigence minimale, naturelle
- représente : $(a \rightarrow b) \Leftarrow D(a) < D(b)$ plus difficile à obtenir

Définition d'une horloge logique

- **H** : ensemble des événements de l'application, muni de l'ordre partiel $\rightarrow$
si ($e \not\rightarrow e'$ et $e' \not\rightarrow e$) alors $e \parallel e'$ (concurrency)
- **T** : domaine de temps, muni de l'ordre partiel $<$
- **Horloge logique :**

$$C : H \rightarrow T$$

$$e \rightarrow C(e)$$

tel que

$$e \rightarrow e' \Rightarrow C(e) < C(e')$$

consistance (simple)*

* L'horloge capture la causalité

Causalité et horloges logiques

- On utilise souvent les horloges pour reconstruire (en partie) la relation de causalité
- $C(e) < C(e') \Rightarrow ?$
 - Soit $e \rightarrow e'$
 - Soit $e \parallel e'$
- Si

$C(e) < C(e') \Rightarrow e \rightarrow e'$ (consistance forte)

les horloges *caractérisent* la causalité

Choisir le type d'horloge en fonction des besoins de l'application

Dépendence Causale entre messages

Dépendance causale entre messages

- On considère que *send(m)* et *receive(m)* correspondent à l'émission et la réception d'un message *m* et que les messages doivent respecter la dépendance causale.

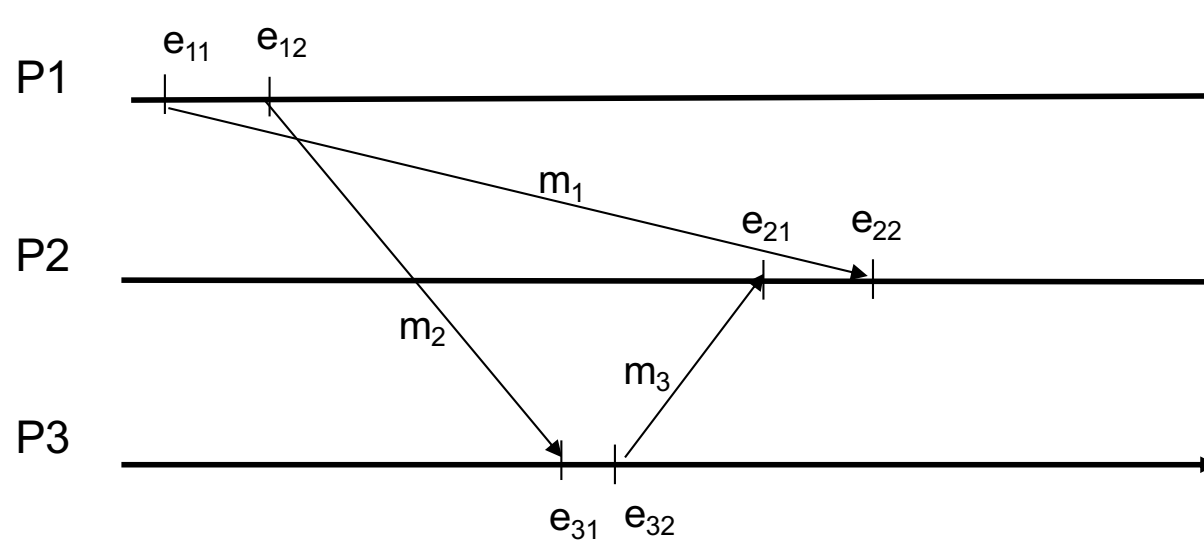
- **Définition :**

Les communications sont ordonnées causalement ("Causal Ordered" - CO), si, pour tout couple de messages *m* et *m'*, on a :

si $send(m) \rightarrow send(m')$ **et** *m* et *m'* ont le même destinataire

alors $receive(m) \rightarrow receive(m')$

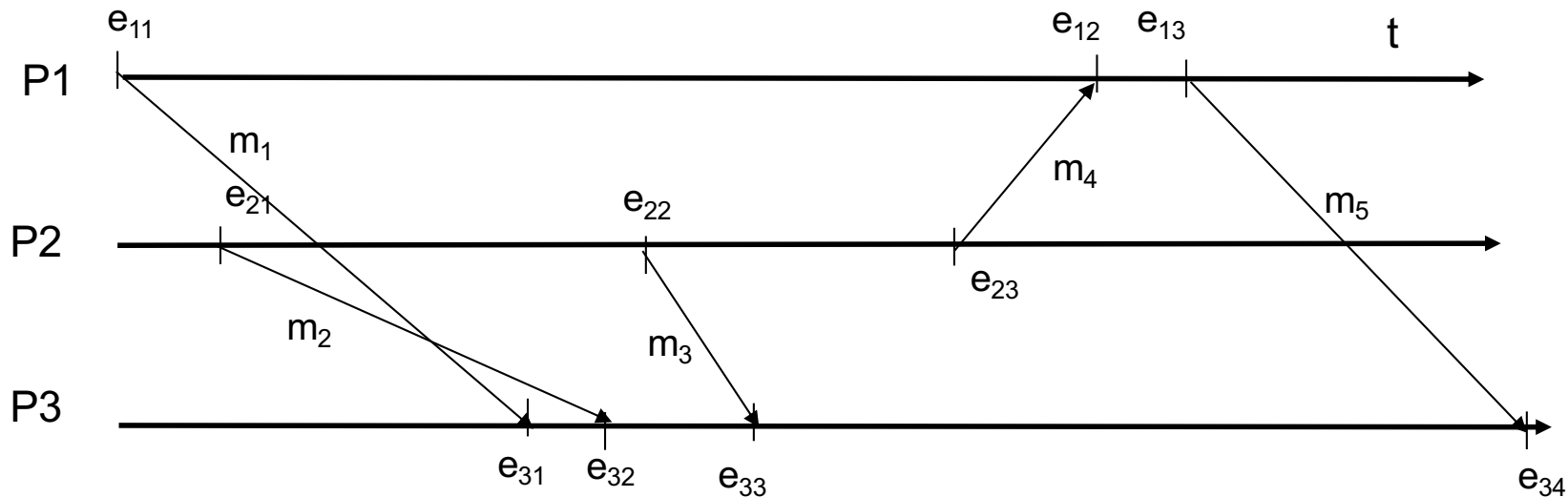
Dépendance causale entre messages : Exemple (1)



La réception sur P2 ne respecte pas la dépendance causale des messages :

$\text{send}(m_1) \rightarrow \text{send}(m_3)$ mais $\text{receive}(m_1) \not\rightarrow \text{receive}(m_3)$

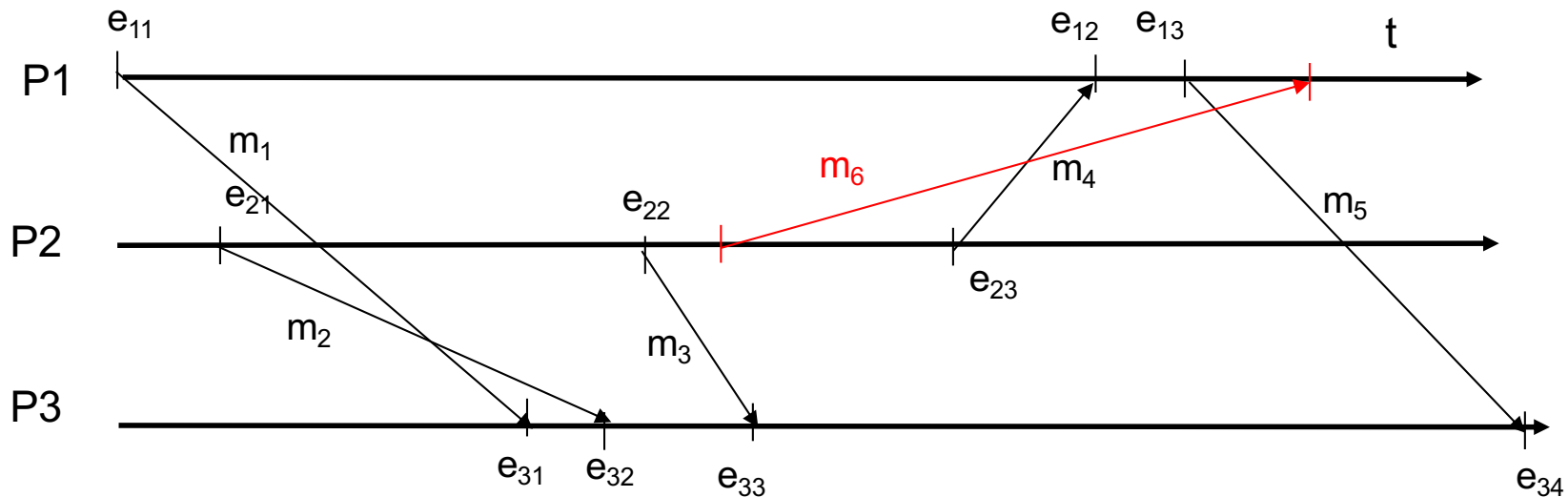
Dépendance causale entre messages : Exemple (2)



En fait, il suffit de ne considérer que les couples de messages ayant le même destinataire :
 (m_1, m_2) , (m_1, m_3) , (m_1, m_5) , (m_2, m_3) , (m_2, m_5) , (m_3, m_5)

- $\text{send}(m_1) \parallel \text{send}(m_2) \Rightarrow$ condition vérifiée
- $\text{send}(m_1) \parallel \text{send}(m_3) \Rightarrow$ condition vérifiée
- $\text{send}(m_1) \rightarrow \text{send}(m_5)$ et $\text{receive}(m_1) \rightarrow \text{receive}(m_5) \Rightarrow$ condition vérifiée
- $\text{send}(m_2) \rightarrow \text{send}(m_3)$ et $\text{receive}(m_2) \rightarrow \text{receive}(m_3) \Rightarrow$ condition vérifiée
- $\text{send}(m_2) \rightarrow \text{send}(m_5)$ et $\text{receive}(m_2) \rightarrow \text{receive}(m_5) \Rightarrow$ condition vérifiée
- $\text{send}(m_3) \rightarrow \text{send}(m_5)$ et $\text{receive}(m_3) \rightarrow \text{receive}(m_5) \Rightarrow$ condition vérifiée

Dépendance causale entre messages : Exemple (2)

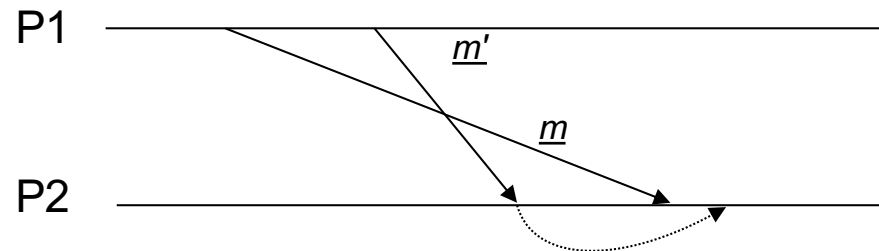


- $\text{send}(m_6) \rightarrow \text{send}(m_4)$ et $\text{receive}(m_4) \rightarrow \text{receive}(m_6)$
 $\Rightarrow$ Dépendance Causale des messages non vérifiée

Délivrance causale

- La **délivrance d'un message m** , notée **$delivre(m)$** , est l'opération qui consiste à rendre m accessible aux applications utilisatrices
- La délivrance d'un message reçu peut être **retardée** lors de sa réception pour garantir un ordre de délivrance souhaité
 - **Ordre de délivrance FIFO**
Si deux messages m et m' sont envoyés successivement depuis un même site $P1$ vers un même destinataire $P2_j$, le premier est délivré sur le site $P2$ avant le second :

$send(m) \rightarrow send(m') \Rightarrow delivre(m) \rightarrow delivre(m')$



Pour respecter l'ordre FIFO la délivrance de m' doit être retardée jusqu'à ce que le message m arrive et soit délivré

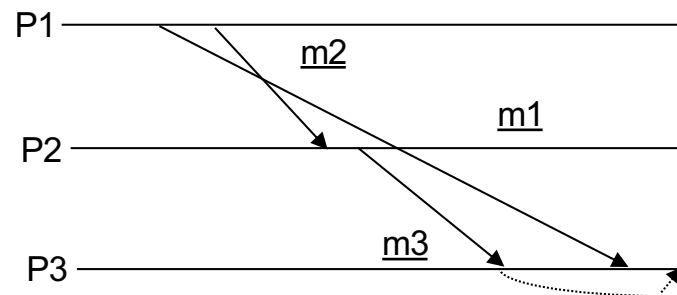
La couche de communication dispose d'un tampon ordonné permettant de mettre les messages en attente

Délivrance causale

- La délivrance causale généralise la délivrance FIFO entre toute paire de processus. Elle correspond à la dépendance causale des messages jusqu'à la délivrance.
- **Définition :**

Les délivrances de messages sont ordonnées causalement (Causaly Ordered Delivery - COD), si, pour tout message m et m' , on a :

si $send(m) \rightarrow send(m')$ **et** m et m' on le même destinataire
alors $delivre(m) \rightarrow delivre(m')$

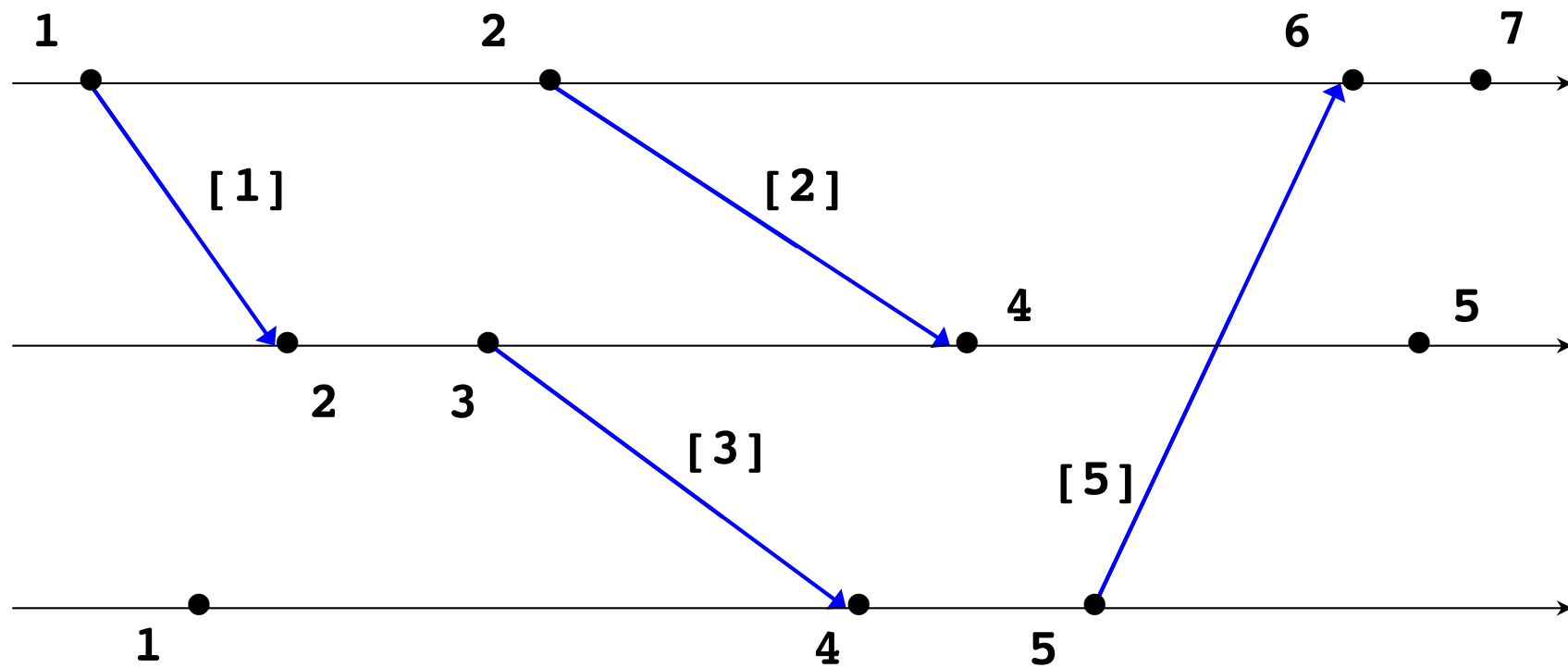


Horloges Scalaire

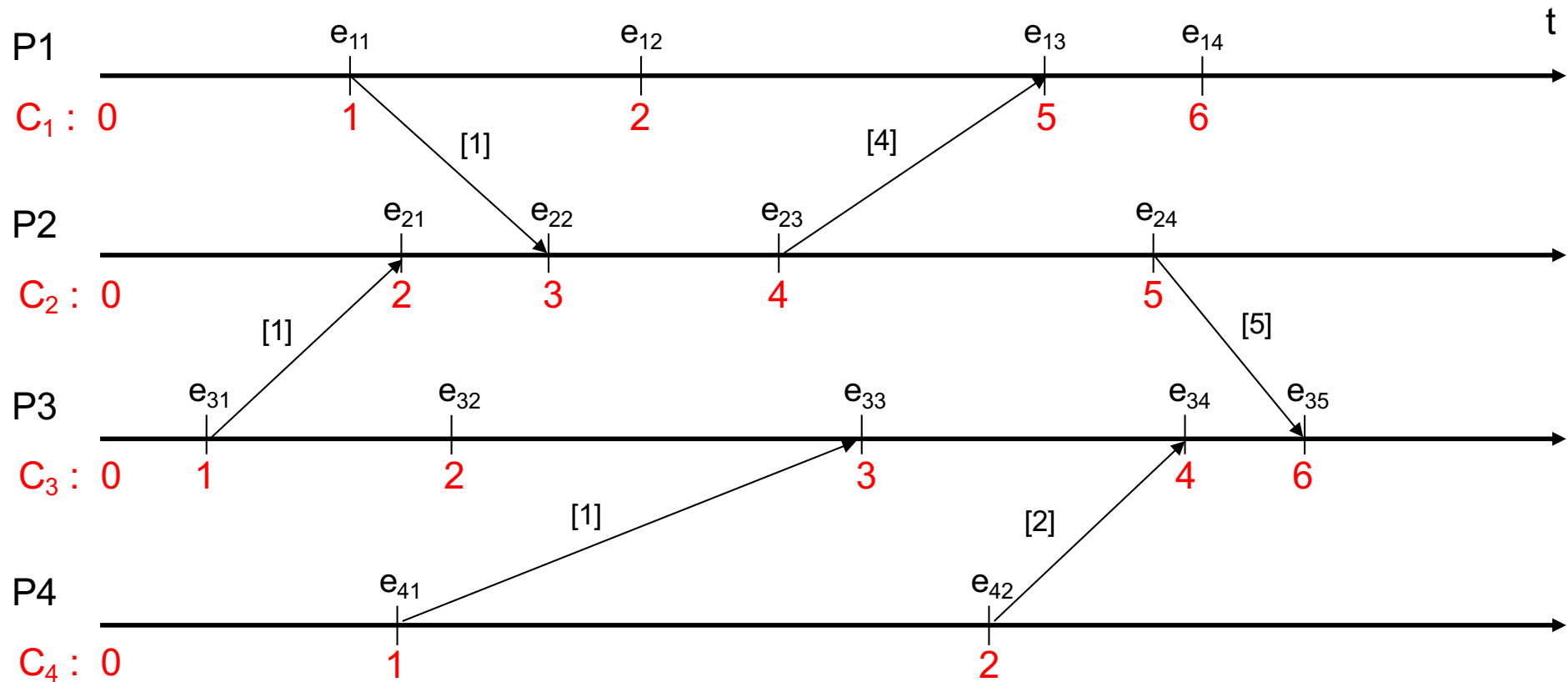
Horloges scalaires (Lamport [78])

- T = entiers naturels N
- Tous les messages portent l'horloge de leur émetteur à l'instant d'émission (**estampillage**)
- **2 règles de mise à jour :**
 - **R1** : avant tout événement (interne, émission, réception), le processus i exécute
 - $C_i = C_i + d$ ($d > 0$, généralement $d = 1$)
 - **R2** : lorsque le site i reçoit un message portant une estampille C_{msg} , il exécute
 - $C_i = \max(C_i, C_{msg})$
 - Appliquer R1
 - Délivrer le message ➤ événement « réception »

Exemple



Horloges scalaires : exemple



Horloges scalaires : propriétés

- C respecte la causalité : $(a \rightarrow b) \Rightarrow C(a) < C(b)$
 - vrai pour deux événements locaux
 - vrai pour une émission et une réception
 - vrai dans tous les cas par induction le long du chemin causal
- C capture la causalité mais ne la caractérise pas : $C(a) < C(b) \not\Rightarrow a \rightarrow b$
 - . exemple : $C(e_{12}) = 2 < C(e_{23}) = 4$ et pourtant $e_{12} \parallel e_{23}$
- **$C(e) = n$** indique que **$(n - 1)$** événements se sont déroulés séquentiellement avant **e** .

Ordre total des événements

- Deux événements peuvent avoir la même date logique (ils sont alors concurrents).
- Horloges scalaires peuvent être utilisées pour définir un **ordre total** $\ll$ sur les événements
- Pour ordonner totalement les événements, on complète la date logique d'un événement par le numéro du processus où il s'est produit : $D(e) = (C_i, i)$, et on utilise la relation d'ordre :

$$e_i \ll e_j \Leftrightarrow C(e_i) < C(e_j) \text{ ou } [C(e_i) = C(e_j) \text{ et } i < j]$$

Exemples utilisation :

- Exclusion mutuelle répartie (la datation logique permet d'ordonner totalement les requêtes pour garantir la vivacité de l'algorithme).
- Diffusion fiable et totalement ordonnée

Horloges Vectorielles

Horloges vectorielles : définition

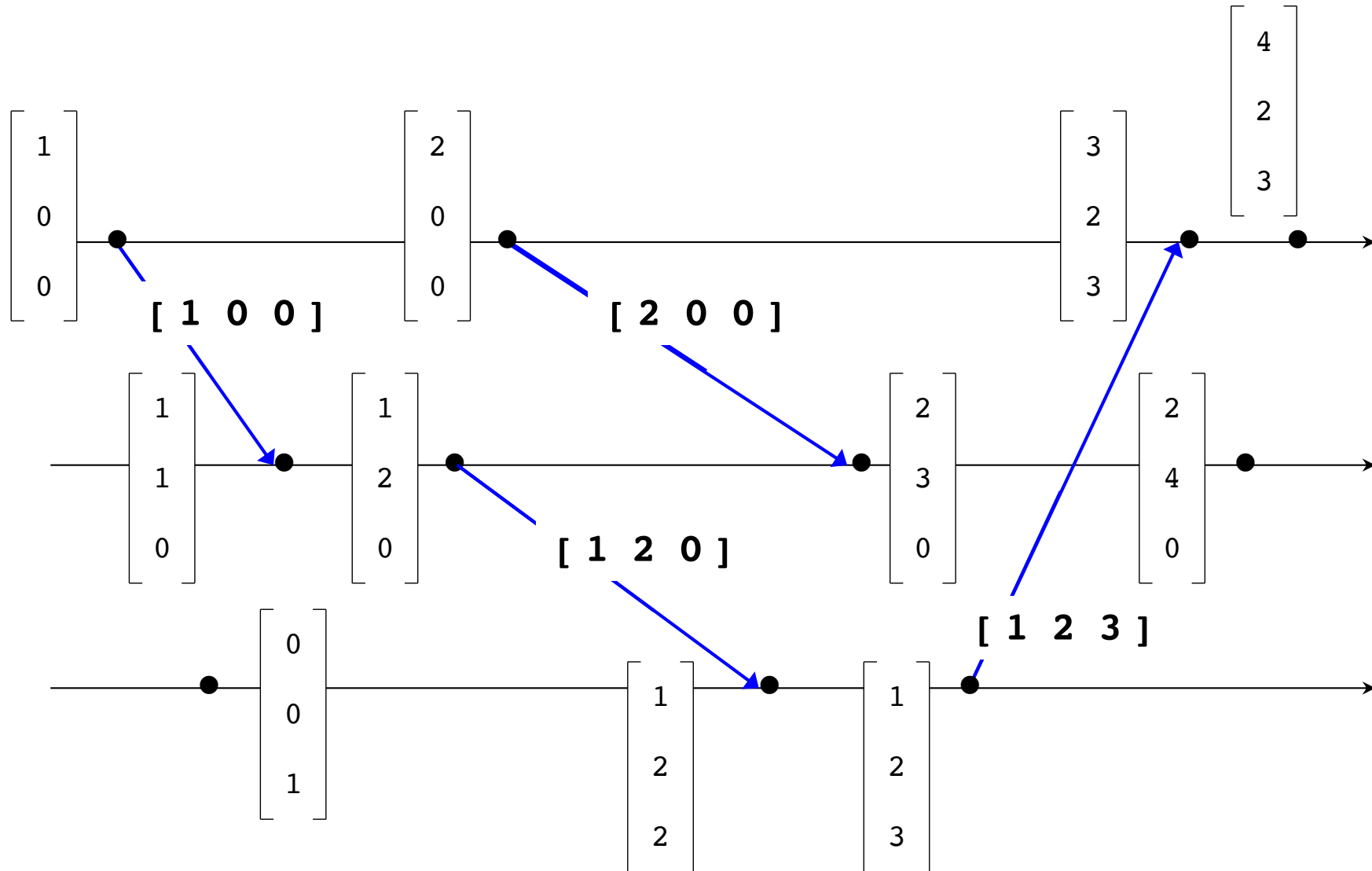
Fidge, Mattern [88]

- Supposent un nombre fixe N de processus
- $T = \mathbb{N}^N$
- Chaque processus i gère un vecteur VC_i de taille N
 - $VC_i[i]$: nombre d'événements du processus i
 - $VC_i[j]$: connaissance qu'a i de l'avancement de l'horloge de j
- A tout instant, l'état réel d'avancement du système est donné par
$$W = (VC_1[1], \dots, VC_i[i], \dots, VC_N[N])$$
mais aucun processus ne peut l'obtenir...

Horloges vectorielles : algorithme de mise à jour

- **Pour le processus P_i :**
 - A chaque événement (émission, réception ou événement interne) de P_i , incrémentation de $VC_i[i]$:
 - $VC_i[i] = VC_i[i] + 1$
 - A l'émission d'un message m :
 - Le message porte la valeur courante de VC_i
 - A la réception d'un message m portant une horloge $m.VC$:
 - Mise à jour de l'horloge courante :
$$\forall x, \quad VC_i[x] = \max (VC_i[x], m.VC[x]) \text{ et } VC_i[i] = VC_i[i] + 1$$

Example



Horloges vectorielles et causalité

- **Les relations suivantes sont définies :**

$$VC_i \leq VC_j \Leftrightarrow \forall k \ VC_i[k] \leq VC_j[k]$$

$$VC_i < VC_j \Leftrightarrow VC_i \leq VC_j \text{ et } VC_i \neq VC_j$$

$$VC_i \parallel VC_j \Leftrightarrow \neg (VC_i \leq VC_j) \text{ et } \neg (VC_j \leq VC_i)$$

- **Les horloges vectorielles définissent un ordre partiel sur les événements**

- tous les vecteurs ne sont pas comparables
- $V \parallel V'$ dénote 2 vecteurs V et V' non comparables

- **Les horloges vectorielles caractérisent la causalité (consistance forte) :**

- Soit $VC(e)$ l'horloge vectorielle d'un événement e

$$e \rightarrow e' \quad \Leftrightarrow \quad VC(e) < VC(e')$$

$$e \parallel e' \quad \Leftrightarrow \quad VC(e) \parallel VC(e')$$

Simplification du test de causalité

- Si e a lieu sur le site i , e' sur le site j :

$$e \rightarrow e' \quad \Leftrightarrow \quad VC(e)[i] < VC(e')[i]$$

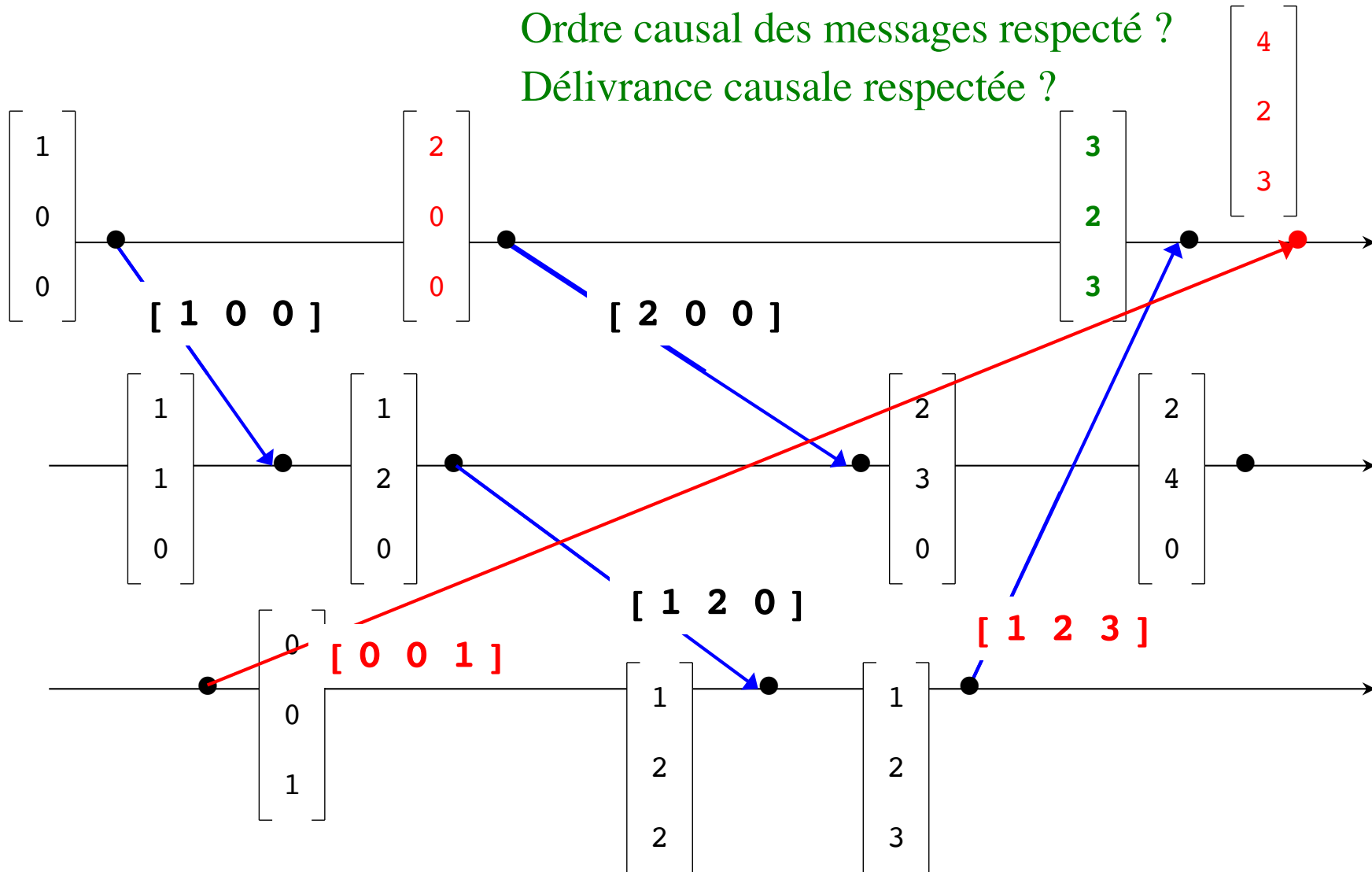
$$e \parallel e' \quad \Leftrightarrow \quad VC(e)[i] > VC(e')[i] \text{ et } \\ VC(e')[j] > VC(e)[j]$$

Horloges vectorielles : LA solution ?

- **Caractérisent la causalité mais...**
 - Augmentent la quantité d'information gérée localement
 - Augmentent l'information circulant sur le réseau
 - Fonction du nombre de processus
 - Taille minimum : Charron-Bost a prouvé que la causalité des événements d'un système réparti avec N processus ne peut être caractérisée qu'avec des horloges vectorielles ayant au moins N entrées.
 - Détection à la volée du respect de l'ordre causale impossible et donc délivrance causale non respectée

Exemple

Ordre causal des messages respecté ?
Délivrance causale respectée ?



Horloges vectorielles : LA solution ?

- **Caractérisent la causalité mais...**
 - Augmentent la quantité d'information gérée localement
 - Augmentent l'information circulant sur le réseau
 - Fonction du nombre de processus
 - Taille minimum : Charron-Bost a prouvé que la causalité des événements d'un système réparti avec N processus ne peut être caractérisée qu'avec des horloges vectorielles ayant au moins N entrées.
 - Détection à la volée du respect de l'ordre causale impossible
- **Peut-on arriver au même résultat en manipulant moins d'information ?**
 - On ne peut pas diminuer l'information gérée localement
 - On peut optimiser l'information envoyée sur le réseau

Horloges Matricielles

Horloges Matricielles

- **M.J. Fisher, A. Michael [82]**
- **$T = \mathbb{N}^{N^2}$**
- **Principe :**
 - Chaque processus gère une matrice MT_i de taille N^2
 - $MT_i[i, i]$: nombre d'événements du processus i
 - $MT_i[i, *]$: nombre de messages envoyés par i aux autres processus
 - Diagonale de MT_i : nombre d'événements locaux des autres sites dont i à la connaissance = horloge vectorielle du processus i
 - $MT_i[j, k]$: nombre de message émis par j à k dont i a connaissance
- Ainsi sur le processus i , à la réception d'un message m en provenance d'un processus j , la valeur de $m.MT[j, k]$ donne le nombre de messages envoyés par j à k et délivrables sur k (c'est à dire dont l'envoi est causalement antérieur à tout ce qui arrivera dorénavant sur i).

Algorithme de mise à jour

- **Pour le processus i**

- A chaque événement local :

$$Mt_i[i, i] = Mt_i[i, i] + 1$$

- A chaque émission d'un message m vers j :

$$Mt_i[i, i] = Mt_i[i, i] + 1 \text{ et } Mt_i[i, j] = Mt_i[i, j] + 1$$

le message porte Mt_i

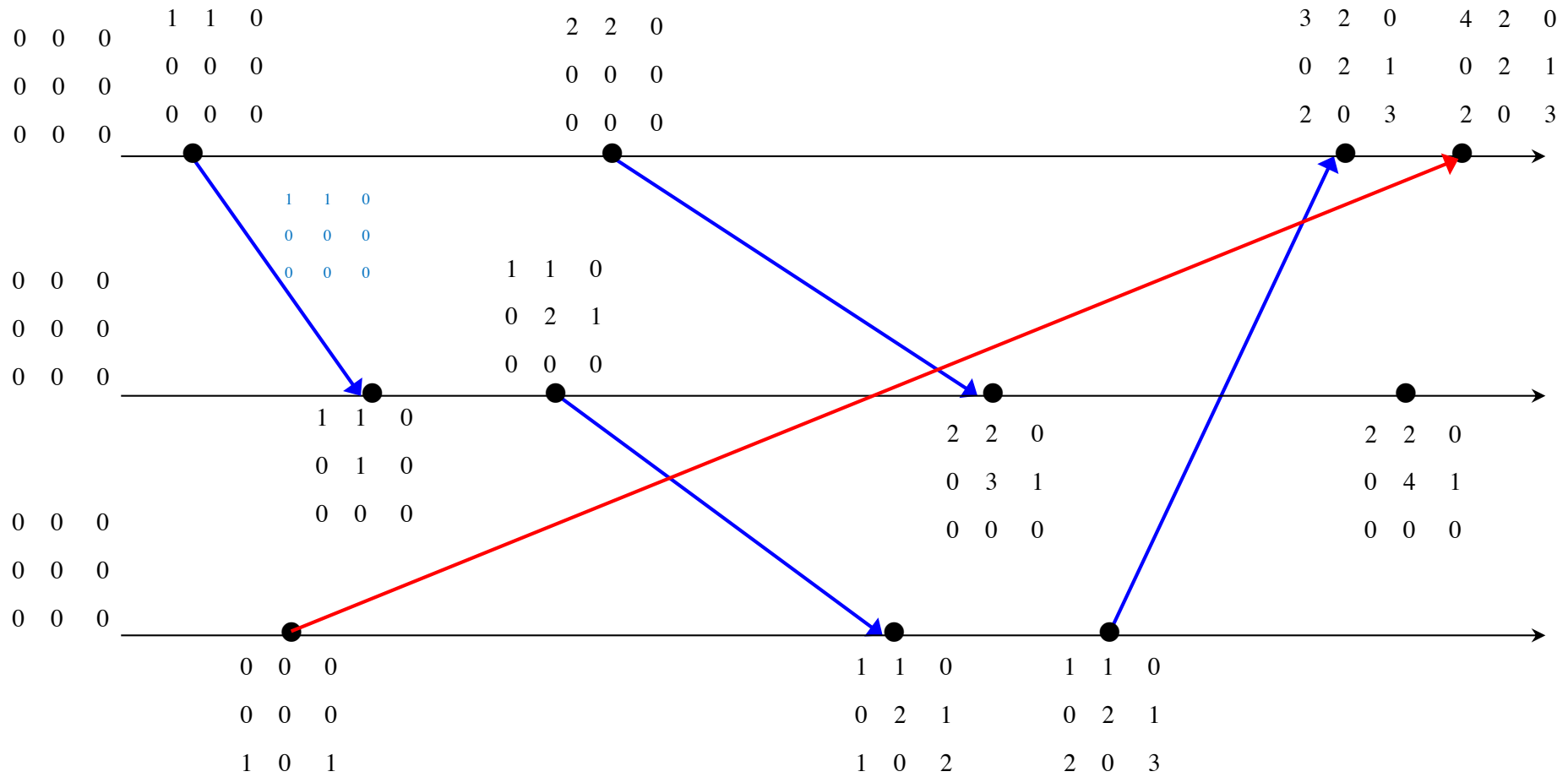
- A chaque réception d'un message m émis par j , il faut s'assurer que:

- tous les messages envoyés antérieurement par j à i ont été reçus
- tous les messages reçus par i et dont l'envoi de m dépend causalement ont bien été reçus

- Les conditions ci-dessus correspondent à vérifier à la réception d'un message m :

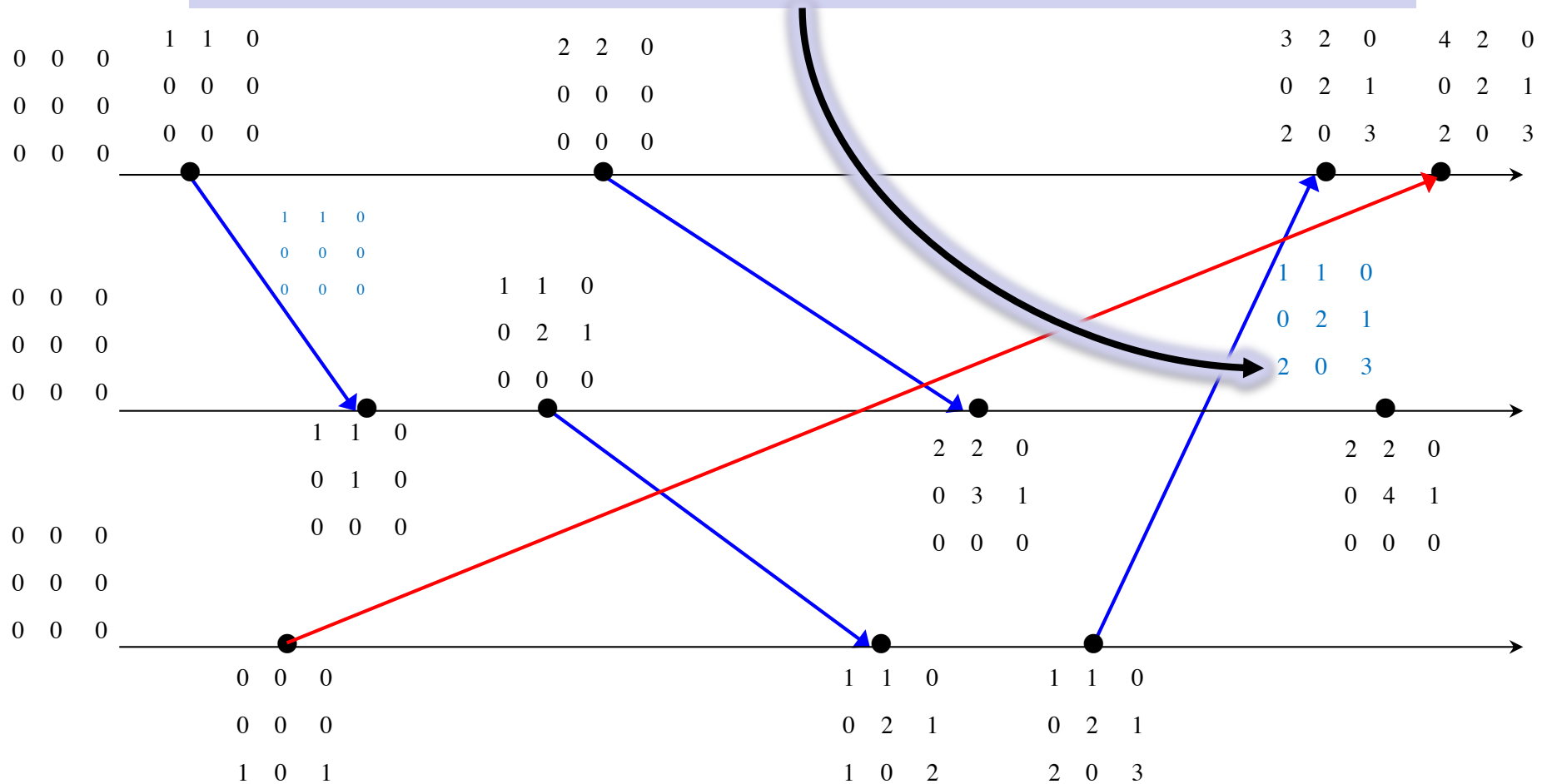
- $m.Mt[i, j] = Mt_i[i, j] + 1$ (ordre causal FIFO respecté en j et i)
- Pour tout $k \notin \{i, j\}$, $m.Mt[k, i] \leq Mt_i[k, i]$

Exemple



Exemple

Ici, à la réception de la matrice venant de P3, P1 voit que P3 lui a envoyé $x = 2$ messages ($M[3,1]=2$), mais que c'est le premier qu'il reçoit ! Il peut donc différer la délivrance du message jusqu'à ce qu'il reçoivent les $(x-1)$ messages manquants (ici, un seul message).



Considérations sur l'approche

- A toutes les propriétés des vecteurs d'horloges
- Très couteux en espace, sur les messages et en calcul
- Processus i sait que tous les autres processus p_j savent que le temps local de chaque p_k a progressé jusqu'au moins t , c'est-à-dire :

$$\min (Mt_i[k, l]) \geq t$$

- Permet d'implémenter un mécanisme tampon (file de messages) pour respecter l'ordre causal des messages à la volée

Conclusions

- **La causalité est la relation qui permet d'analyser une application répartie**
 - Pas d'exécutions identiques
 - Nécessité de tracer le lien entre les événements
- **Le type d'horloge dépend des besoins de l'application**
 - Consistance forte ? Horloges vectorielles
 - Consistance simple ? Horloges scalaires
 - Ordre causal des messages ? Horloges matricielles

Bibliographie

- Lamport, L. *Time, clocks, and the ordering of events in a distributed system*. Communications of the ACM, 21(7), 1978.
- Mattern, F. *Virtual Time and Global States of Distributed Systems*. In Proceedings. Workshop on Parallel and Distributed Algorithms, 1989.
- C. Fidge. *Logical time in distributed computing systems*. IEEE Computer, pages 28-33, July 1991.
- M. Singhal, A. Khsemkalyani. *An efficient implementation of vector clocks*, Information Processing Letters, 992, vol. 43, no1, pp. 47-52.
- M. Raynal and M. Singhal. *Logical time: a way to Capture Causality in Distributed Systems*. Technical Report n. 2472, INRIA - Rennes, 1995
- Bernadette Charron-Bost: *Concerning the Size of Logical Clocks in Distributed Systems*. Information. Processing Lett. 39(1): 11-16 (1991)